Prueba tecnica mid/senior

Laravel / MySQL / Frontend / Debugging / Trazabilidad

Duracion estimada	90 minutos
Entrega	Repositorio de GitHub con instrucciones claras de ejecucion
Stack principal	PHP + Laravel + MySQL
Frontend	Blade/Livewire o Inertia + React. Puede usarse Bootstrap o Tailwind. No usar Next.js.

1. Contexto

Queremos comprobar que sabes construir una funcionalidad pequena, pero realista, con el stack indicado; que sabes modelar datos con criterio, mantener trazabilidad, localizar inconsistencias y proponer correcciones seguras. La prueba no parte de un repositorio preparado. Debes crear el proyecto, organizarlo y dejar instrucciones suficientes para que podamos ejecutarlo.

La aplicacion a construir se llama: Panel operativo de incidencias

No se busca una aplicacion grande. Se busca una solucion pequena, coherente, mantenible y razonada. Para este perfil se valorara especialmente el criterio: transacciones, validaciones, relaciones, trazabilidad y claridad ante errores de datos.

2. Stack obligatorio

- PHP + Laravel.
- MySQL como base de datos.
- HTML y CSS.
- Bootstrap o Tailwind para apoyo visual.
- JavaScript para interaccion de interfaz.
- Frontend con Blade/Livewire o con Inertia + React. No usar Next.js.

Puedes usar IA para ayudarte, pero debes documentar brevemente como la has utilizado. El objetivo no es prohibir herramientas, sino demostrar que entiendes, revisas y controlas lo que entregas.

3. Ejercicio 1 - Maquetacion de la pantalla principal

Crea una pantalla principal para consultar y operar incidencias. Debe incluir:

- Cabecera con el nombre de la aplicacion y una breve descripcion funcional.
- Listado de incidencias en tabla o tarjetas, con prioridad, estado, fecha y responsable o solicitante.
- Indicadores visuales claros para los estados: abierta, en revision, bloqueada, resuelta.
- Formulario para crear una nueva incidencia.
- Accion visible para cambiar el estado de una incidencia.

- Acceso al historial de cambios de una incidencia.
- Diseno limpio, responsive y usable en escritorio y movil.

Para un perfil mid/senior no basta con que la pantalla se vea correcta: debe ser facil de entender, mantener y ampliar. Evita CSS caotico, componentes gigantes y markup innecesariamente acoplado.

4. Ejercicio 2 - Base de datos y modelo de datos

Define las migraciones necesarias para trabajar, como minimo, con estas entidades:

Tabla	Campos minimos	Descripcion
incidents	id, title, description, status, priority, requester_name, assigned_to, created_at, updated_at	Incidencias registradas.
incident_logs	id, incident_id, action, old_value, new_value, user_name, metadata, created_at	Trazabilidad de cambios relevantes.
incident_comments	id, incident_id, author_name, body, created_at, updated_at	Comentarios operativos de seguimiento.

Requisitos adicionales del modelo:

- Usar claves foraneas donde corresponda.
- Definir indices razonables para consultas habituales por estado, prioridad y fecha.
- Evitar strings magicos dispersos: centraliza los estados de forma clara.
- Validar que los cambios de estado sean coherentes.
- Guardar los cambios criticos en incident_logs.
- Usar transacciones cuando una accion modifique la incidencia y su trazabilidad asociada.

5. Ejercicio 3 - Programacion funcional minima

Implementa las siguientes acciones:

- Crear una incidencia con validacion de datos.
- Listar incidencias con filtros por estado, prioridad y texto libre.
- Cambiar el estado de una incidencia respetando transiciones permitidas.
- Registrar en incident_logs cada cambio de estado, incluyendo valor anterior, valor nuevo, usuario o actor y fecha.
- Mostrar el historial de cambios y comentarios de una incidencia.
- Anadir un comentario operativo a una incidencia.
- Gestionar errores de validacion de forma clara para el usuario.

Transiciones minimas esperadas: abierta -> en revision -> bloqueada o resuelta. Una incidencia resuelta no deberia volver a abierta sin dejar log explicito de reapertura.

La solucion puede implementarse con controladores clasicos, Livewire o Inertia + React. Elige una aproximacion y mantenla coherente.

6. Ejercicio 4 - JavaScript / React / interaccion

Anade interaccion de frontend que mejore la experiencia de uso. Debe incluir al menos dos de estas funcionalidades:

- Filtro rapido por estado sin recargar toda la pagina.
- Confirmacion antes de resolver o bloquear una incidencia.
- Contador visual de incidencias por estado.
- Resaltado de incidencias vencidas o bloqueadas.
- Componente React o Livewire para filtrar, actualizar o resaltar incidencias.

No se pide una arquitectura frontend compleja, pero si orden. Se debe poder entender donde vive el estado, donde se dispara la accion y como se refresca la interfaz.

7. Ejercicio 5 - Diagnostico y correccion de inconsistencia de datos

Simula en tu propio proyecto una incidencia tipica de mantenimiento y deja documentada la correccion en el README.

Caso a resolver:

- Crea datos de ejemplo con al menos tres inconsistencias: una incidencia resuelta sin log de resolucion, una incidencia cuyo estado no coincide con su ultimo log, y un log duplicado o incoherente.
- Detecta esas incidencias mediante SQL o Eloquent.
- Implementa una forma de diagnostico clara: comando Artisan, clase de servicio, script documentado o consulta claramente identificable.
- Corrige los datos de forma segura, explicando si creas logs faltantes, ajustas estados o marcas inconsistencias para revision manual.
- Incluye una opcion dry-run o una explicacion equivalente que permita ver que se corregiria antes de modificar datos.
- Documenta la query usada y explica brevemente por que detecta el problema.

Se espera prudencia. No todas las inconsistencias deben arreglarse automaticamente si no hay informacion suficiente. Si decides no corregir una de ellas de forma automatica, explicalo.

8. Uso de IA

Puedes usar herramientas de IA para desarrollar la prueba. Si las usas, anade en el repositorio una carpeta llamada:

`/ai-process`

Incluye dentro un archivo breve con:

- Herramientas utilizadas.
- Prompts principales o resumen de los prompts.
- Partes generadas con ayuda de IA.
- Cambios, correcciones o decisiones tomadas manualmente.
- Errores detectados en la salida de la IA y como los corregiste.

No hace falta escribir una novela. Basta con demostrar que has dirigido y revisado el proceso.

9. Entrega

Entrega un repositorio de GitHub que incluya:

- Codigo fuente del proyecto.
- Migraciones y seeders necesarios.
- README con instrucciones para instalar y ejecutar.
- Archivo .env.example sin credenciales reales.
- Explicacion breve de las inconsistencias simuladas y de como las has diagnosticado/corregido.
- Documentacion de uso de IA, si aplica.

10. README minimo esperado

El README debe permitir ejecutar el proyecto sin adivinar. Incluye como minimo:

- Requisitos necesarios: version de PHP, Composer, Node y MySQL.
- Pasos de instalacion.
- Comandos para migrar y cargar datos de ejemplo.
- Comando para levantar la aplicacion.
- Comando o pasos para ejecutar el diagnostico de inconsistencias.
- Usuario de prueba si has creado autenticacion. La autenticacion no es obligatoria.
- Notas sobre decisiones tecnicas.

11. Alcance recomendado

La prueba debe seguir siendo acotada. No es necesario implementar:

- Autenticacion completa.
- Roles y permisos avanzados.
- Notificaciones.
- Tests extensos.
- Docker.
- Deploy publico.
- Integraciones externas.

Es preferible una solucion pequena, consistente y ejecutable que una aplicacion grande incompleta. Para este nivel, la diferencia esta en el criterio tecnico, no en la cantidad de pantallas.